

Safety Patrol Done

Team Members:

Student Name	Github
Hector Freard Ruiz	https://github.com/Hectorfr
Florian Thollot	https://github.com/Flo822
Ondrej Duba	https://github.com/Ondralol

Date: 24/05/2026

Table of Contents

Abstract.....	3
1. Introduction and Background.....	4
2 Overview of the architecture/system	5
3 Results and Evaluation	5
4 Discussion and Conclusions.....	6
5 References	6
Appendices	7

Abstract

We created an autonomous patrol drone system whose primary purpose is to detect potentially hazardous objects inside warehouses and factories. These detected objects are reported via our application interface so they can be disposed of appropriately before causing injury. The drone flies on a predefined path and uses its camera to detect these hazardous objects. When such an object is detected, we approximate its position, display it on the map, and save the timestamp, object type, and coordinates in a log. A typical use case for our system is deploying the drone after a work shift ends, so that any dangerous objects left behind can be detected and cleaned up before the next shift starts. Examples of such objects include liquid spills and work tools.

1. Introduction and Background

In large warehouses and factories, safety is a high priority, but achieving good safety is not always easy. One of the many safety risks is the presence of hazards on the floor. These hazards include liquid spills (water, oil, etc.) and misplaced tools such as hammers, screwdrivers, and more. All these hazards can potentially cause injuries. Inspections of the work areas are typically done manually, which is time-consuming and can be inconsistent. In order to improve the efficiency and quality of these inspections, we created a system that utilizes an autonomous drone with a camera that provides a live video feed, which is then used to detect these objects. For each detected object, we not only display the object type, location, and time of detection, but also log this information locally for further analysis. Since we are limited by the hardware, our application serves more as a proof-of-concept. We use the DJI Tello EDU drone (Ryze Technology, 2019), which is a lightweight drone with a 720p 30FPS camera and a very limited number of sensors. However, the core of the application can be easily applied to other, more professional drones.

1.1 The problem you tried to solve

In large factories, warehouses, and other large work areas, it is difficult to maintain a safe environment for workers. Even though safety should be the highest priority, leftover tools and other safety risks are often left behind and can not only cause injuries to individuals but also slow down the production by creating inefficiencies. Manually looking for such hazard requires somebody to look around the whole workplace area and identify these risks. This activity is time-consuming, inconsistent, and quite frankly, often skipped or done improperly.

1.2 Motivation

We are proposing an autonomous drone solution for detecting hazard in order to improve the safety of large work areas. This will create safer work environments, saving workers from injuries and it could directly lead to more efficient working conditions. Our solution also provides a faster alternative to manual inspections and since the drones are generally cheaper than workers, it's also a financially attractive alternative.

1.3 Application

We created a desktop application written in Python that contains all the necessary components for the drone inspection. It contains the drone control, machine vision pipeline, and the visualisation and logging of the detected objects. The application provides advanced, but easy to use user interface that allows the operator to easily control and debug the drone (this includes starting the automatic sequence and manual control of the drone), and visualisation of the detected objects. There are additional features such as the status bar that shows telemetry information of the drone, the map, that

shows the drone's current position as well as the position of detected objects, and the log that shows all of the detected objects in structured format with timestamps and coordinates.

1.4 Dataset

We decided to use two datasets. The first dataset that we used was Objects365 (Shao et al., 2019), which contains 365 object categories, over 600K images, and more than 10 million labels. The second dataset we used was the Spill Detection Dataset (Forero, 2023), which contains over 5000 images of different liquid spills. In the Spill Detection Dataset, there are 6 different spill categories. For this proof-of-concept application, we decided to use only 6 classes. From the Objects365 dataset, we filtered the classes knife, hammer, tape, screwdriver, and scissors. From the Spill Detection dataset, we filtered for 2 spill classes and combined them into a single spill class. For each filtered class, we obtained between 1500 and 2500 images, and in total, we ended up with a dataset containing 10,504 images and 6 classes. The dataset was constructed in Roboflow (Roboflow, 2026), which allowed us to filter images directly rather than downloading all images and filtering the data manually. We further used Roboflow to create Train, Validation, and Test splits directly on the site, and resized all images to 640x640. The splits were set to 70% for the training dataset and 15% each for the validation and test datasets. This final dataset (Duba, et al.) is publicly available and can be accessed and reused.





(Train dataset image showcase with labels and bounding boxes)

2 Overview of the architecture/system

Our architecture consists of two parts: the Drone and the actual application. Before starting the application, we connect to the drone's Wi-Fi network, and upon launching the application, we connect to the drone by sending a connection command. This allows us to then further communicate with the drone.

The drone's part of the workflow is sending telemetry commands, such as current temperature, current height, or battery temperature, and primarily, the video stream. The drone then executes commands we send from the application, such as where to move or how to rotate. Additionally, we can send commands for taking off, landing, etc.

The application receives the drone's telemetry and video livestream and uses an object detection model to detect objects in each frame. The livestream and detected objects, along with the drone's current position, are visualized in the GUI, and the detected objects are also logged with their class, timestamp, and estimated position. The user can also send commands directly to the drone. The commands that are sent to the drone are either commands sent by the user or commands defined in an automatic sequence.

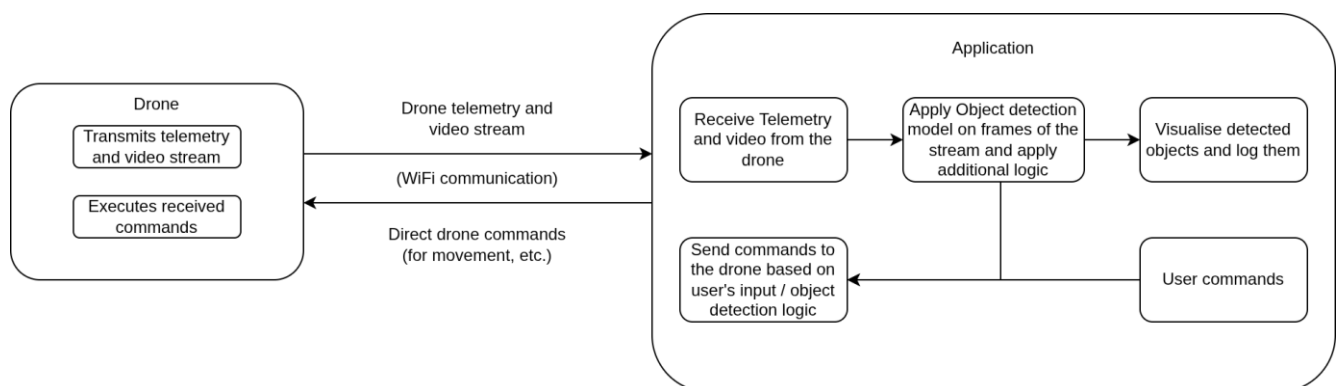
The object detection processing consists of getting the live stream and applying the object detection model. To improve object detection, we added a few enhancements. First, we added a parameter that dictates how often we run the inference on the livestream. Some of the models we used were slow, and running inference on every frame made the application unusable on our hardware. By running inference on every n -th frame (n could be 2-4), we wouldn't lose too much information while dramatically improving performance. Another improvement was to confirm a detected object only if it was detected in the same place for 2 or more consecutive frames. This helps eliminate false positives.

We created two automated movement sequences: the path sequence, which defines a sequence of commands that move the drone across the entire work area, and the inspection sequence, which is used to inspect a detected object more closely. The purpose of the path sequence is to define every movement and rotation that the drone needs to execute in order to inspect every part of the work area. This sequence can be launched by simply pressing one button in the GUI. The drone will start flying and detecting the objects on its own. The inspection sequence was created to get a closer look at the detected objects. The idea is simple: once an object is detected, the drone flies around it in an arc to get a better view from different angles. This allows a better, more consistent object detection. We added an ability for the drone to directly activate the inspection sequence while it is in the path sequence and detects an object, but we had to turn this feature off due to testing area limits. Our testing site was a 2x2x2m area, and there wasn't enough space to properly test the inspection sequence combined with the path sequence. However, we added a button to trigger the inspection sequence manually so it could still be tested.

In order to track the drone's position and visualise it along with the position of the detected objects, we had to use dead reckoning. That means we had to manually track every command we sent to the drone and update the drone's position ourselves based on those commands. The starting coordinates were set to (0, 0, 0), indicating the starting position, and every movement updated the drone's position relative to the starting point. This created a few problems, since the drone's movement was inaccurate, and even though we correctly tracked its position, the drone often drifted away from its position, even when no commands were sent. The movement commands were also inconsistent, and sending the same commands often resulted in different movements. As a result, the mapping of the detected objects was not always accurate.

2.1 Flow Diagram

The workflow consists of the Drone and the Application, which communicate over Wi-Fi. The Drone transmits the telemetry and the video stream and executes commands. The Application receives the video stream, applies an object detection pipeline along with additional logic, and transmits commands that control the drone. Application visualises and logs the detected objects, and provides an interface that allows sending manual commands for direct control, and sequence commands (path/inspection sequence) for automatic control.

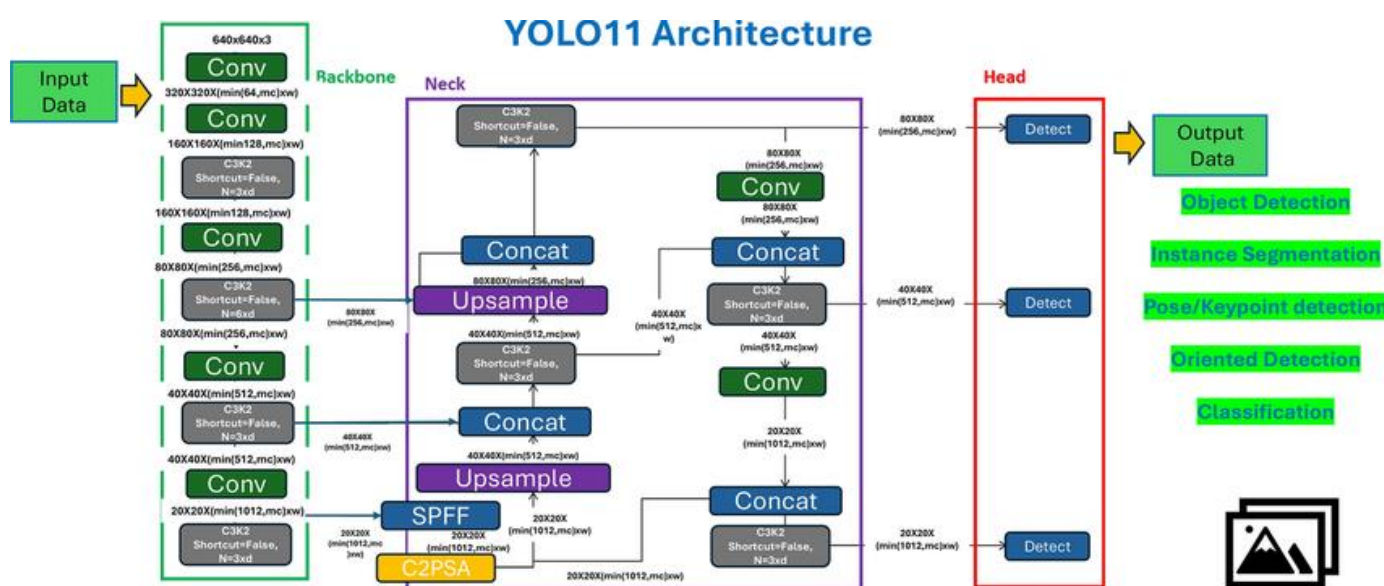


(Workflow diagram of the application and drone)

2.2 CNN Architecture Design

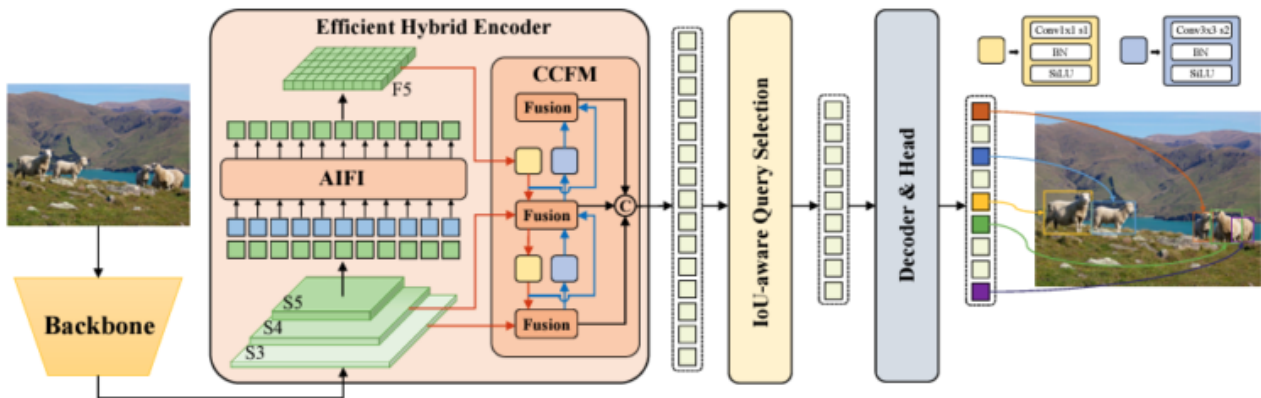
Because we needed to use object detection in real time, we had to use an architecture with fast inference. For that reason, we decided to compare two object detection architectures: You Only Look Once: YOLO (Redmond et al., 2015) and Real-Time DETection Transformer: RT-DETR (Zhao et al., 2023), both of which are very efficient and built for real-time inference.

YOLO is a single-stage object detection model that processes images in a single forward pass, and its initial version was released in 2015. We decided to compare multiple YOLO versions. We compared YOLOv5 Nano (2.6M parameters), YOLO11 Nano (2.6M parameters), YOLO11 Small (9.4M parameters) and YOLO11 Medium (20.1M parameters). In more detail, YOLO11 works by first passing an image through a CNN backbone used as a feature extractor. Then there is a neck that serves as a processing stage, enhancing feature representations. Finally, the head is used for the final prediction, creating outputs for localization and classification (Khanam and Hussain, 2024).



(YOLOv11 architecture diagram. Source: Sapkota et al., 2025)

For the RT-DETR, we used a RT-DETR Large (33M parameters). RT-DETR works by first passing an image through a CNN backbone, which extracts features, more specifically, the features are extracted from the last 3 stages of the CNN. This is then fed into a hybrid encoder that transforms the features into a sequence of image features. IoU-Aware Query selection is then used to select a set of image features from the encoder, which are then fed into a decoder and a head that generate the final predictions and bounding boxes (Rath, S. J., 2024).



(RT-DETR architecture diagram. Source: Rath, S. J., 2024)

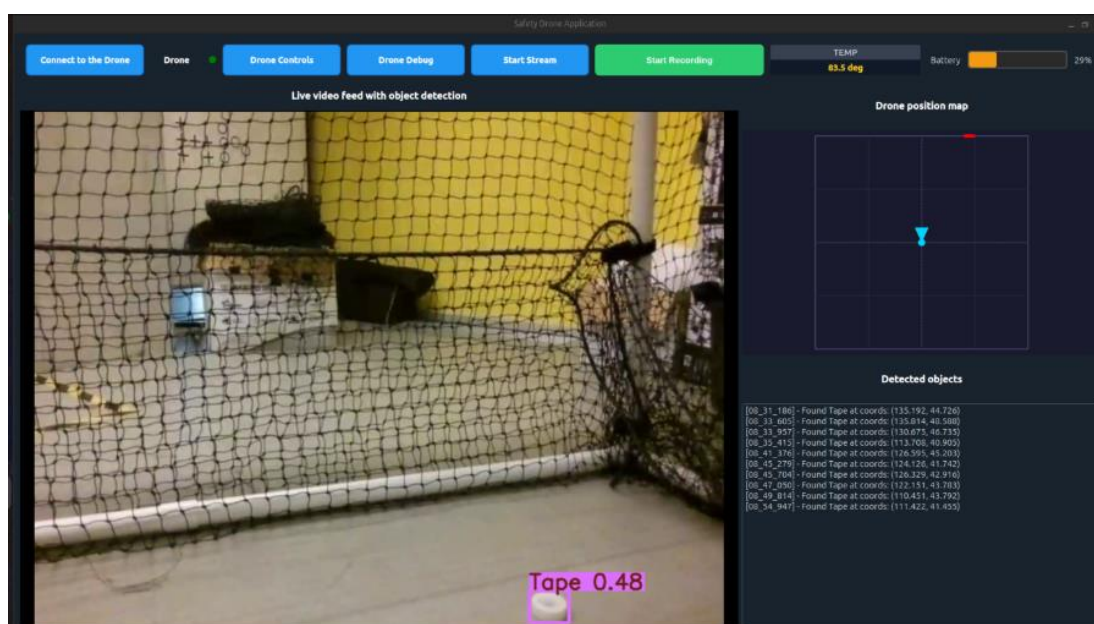
We achieved the best accuracy with RT-DETR Large, but the final model used for the actual drone inspection was YOLO11 Medium. This was because of the hardware limitations of our computers. We weren't able to use RT-DETR properly because the computer on which the application was running had no GPU. However, on computers with GPUs, RT-DETR should be able to run with no issues.

2.3 GUI Design

Our GUI consists of several parts: the Live Video Feed, Drone Map, Drone Log, Status Bar, Drone Control Popup, and Debug Popup.

Full GUI overview

This shows the full application interface.



Live Video Feed

The live video feed shows the drone's live stream, along with object detection inference. That means that if an object is detected, we display the bounding rectangle and the corresponding class.



Object and drone map

On the map, we display the drone's actual position and the direction it is currently facing. This position is relative to the starting point (the middle of the 2x2x2 area where the drone was tested). We also display the approximate positions of all detected objects on this map.



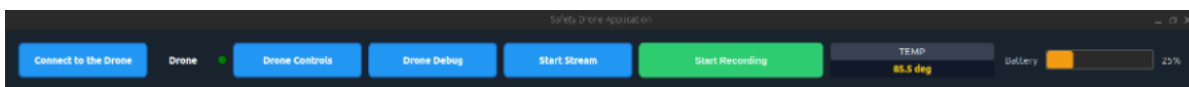
Object log

In the log, we show all detected objects, their positions (relative to the starting point), and the timestamps.

```
[08_31_186] - Found Tape at coords: (135.192, 44.726)
[08_33_605] - Found Tape at coords: (135.814, 48.588)
[08_33_957] - Found Tape at coords: (130.675, 46.735)
[08_35_415] - Found Tape at coords: (113.708, 40.905)
[08_41_376] - Found Tape at coords: (126.595, 45.203)
[08_45_279] - Found Tape at coords: (124.126, 41.742)
[08_45_704] - Found Tape at coords: (126.329, 42.916)
[08_47_050] - Found Tape at coords: (122.151, 43.783)
[08_49_814] - Found Tape at coords: (110.451, 43.792)
[08_54_947] - Found Tape at coords: (111.422, 41.455)
[08_59_121] - Found Tape at coords: (123.861, 43.725)
```

Status bar

In the status bar, there are buttons to connect to the drone, start streaming, and start recording. The recording saves the drone's broadcast along with the inference locally. The status bar also shows in real time whether we are connected to the drone, and once connected, it displays the drone's temperature and battery level.

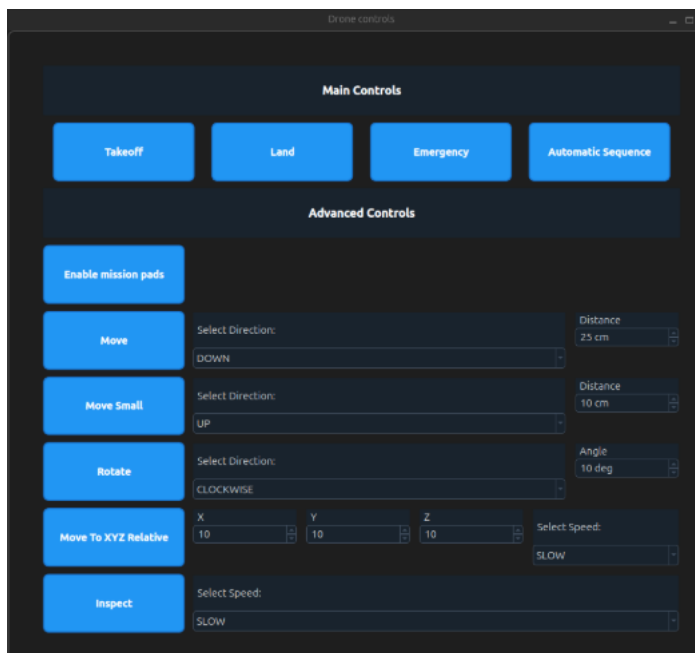


Drone Main Control Popup

The drone Control Popup window can be opened by clicking on the Drone Controls button in the status bar, and it is divided into main and advanced controls.

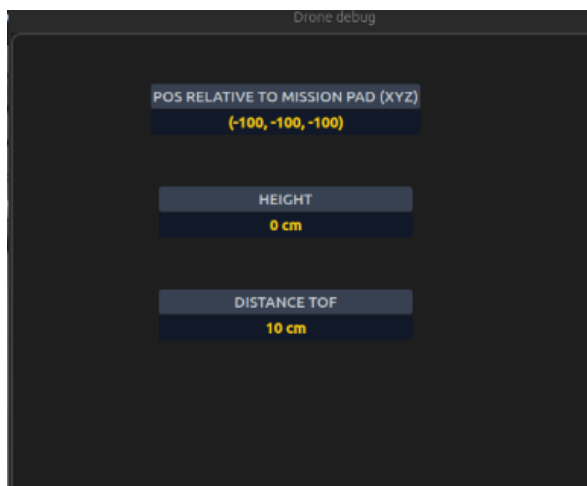
The main controls include buttons that initiate Take-off, Landing, Emergency (which overrides every command and stops the drone immediately), and Automatic Sequence (which initiates the automatic movement path sequence in the designated area).

The advanced controls include enabling the mission pads, which are used for drone debugging (they can be used to track the drone's position relative to them), and several buttons that control the drone's movement. There are two buttons for basic movement: move and move small. Using these buttons, the drone can be manually moved in 6 directions: up, down, left, right, forward, and backward. The difference between move and move small is in the granularity of the movement. In a normal move, the drone performs a single continuous movement, and in a move small, it performs the movement in a sequence of smaller steps. Then there is a button that rotates the drone and a button that allows movement of the drone relative to the current position. For each movement, there are more options, such as speed selection and whether or in which direction the drone should move. Finally, there is an inspection button that initiates an inspection sequence, which is used to perform a symmetric inspection sweep.



Drone Debug Popup

Drone Debug Popup can be opened by clicking on the Debug button in the status bar. It shows more telemetry from the drone, which can be useful for debugging, such as the drone's height and position relative to the mission pads.



3 Results and Evaluation

We compared 2 different architectures, YOLO and RT-DETR, on a dataset containing over 10K images and 6 classes. We experimented with both transfer learning and training from scratch. In total, we conducted 6 experiments with different model sizes and achieved the best accuracy (mAP50) with the RT-DETR Large, followed by YOLO11 Medium. As previously mentioned, YOLO11 Medium was chosen as our final model even though it didn't have the best accuracy.



(Example of YOLO11 Medium inference on the Test Dataset)



(Example of YOLO11 Medium inference on the drone livestream)

3.1 Experimental Settings

We ran a total of 6 experiments. For the YOLO architecture, we compared the YOLOv5 Nano, YOLO11 Nano, YOLO11 Small, and YOLO11 Medium. For all these architectures, we used transfer learning. YOLO Models are pretrained on the MS COCO dataset (Lin et al., 2014), which contains over 328K images and 91 object classes. We also trained a YOLO11 Nano from scratch with no pretrained weights to establish a baseline accuracy. For the RT-DETR Large, we also used transfer learning, and the model was also pretrained on the MS COCO dataset, the same as the YOLO.

For all the YOLO models except YOLO11 Medium, we used the same settings. We set the training to 250 epochs, used an image size of 640x640, and a batch size of 4. The optimizer we used was AdamW with a learning rate of 0.001, weight decay of 0.0001, and cosine learning rate set to true. We also used 3 warmup epochs and a patience of 30. The patience was used to stop training early if there was no improvement in validation accuracy after 30 epochs. We did, however, always check the training curves to ensure the training actually converged. We also used an augmentation. Scale and HSV saturation augmentations were used with 0.3 probability, translation was used with 0.1 probability, mosaic and flip left-to-right were used with 0.5 probability, and HSV value augmentation was used with 0.15 probability.

For YOLO11 Medium and RT-DETR, we trained for only 150 epochs and used patience 30. This was due to the models being much larger and the training taking significantly longer. For both YOLO11 Medium and RT-DETR Large, we used the SGD optimizer with a learning rate of 0.01 and a momentum of 0.9. For the YOLO11 Medium, we also set the starting learning rate to 0.005 and batch size 8, and for the RT-DETR Large, we set the batch size to 64. Both of these models used the same augmentation, which was crop fraction and mosaic with probability of 1.0, erasing and HSV value augmentation both with probability of 0.4, horizontal and vertical flips, and scale all with probability 0.5, translation with probability 0.1, HSV hue augmentation with 0.015 probability, and HSV saturation augmentation with 0.7 probability.

3.2 Pre-Processing (If Applicable)

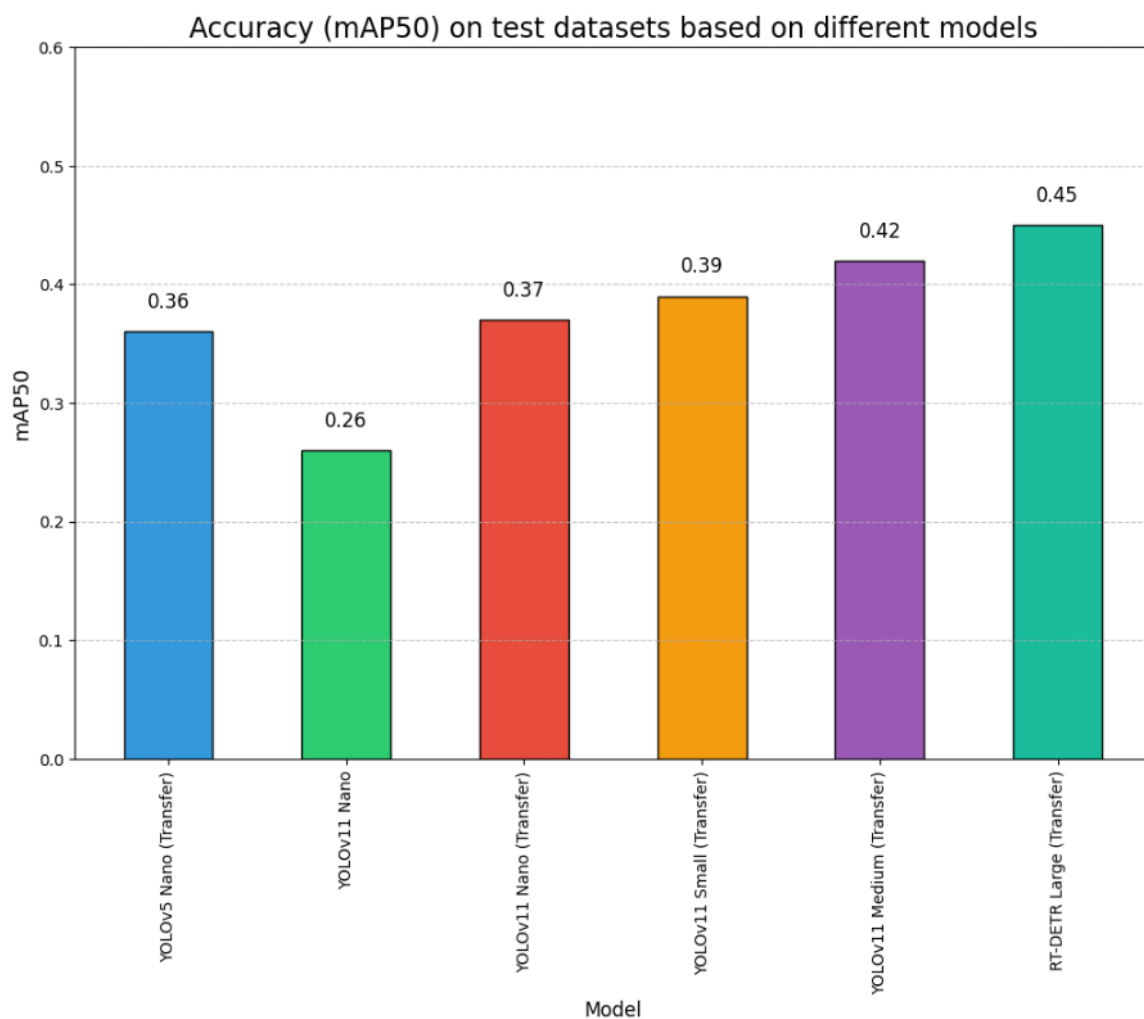
The only preprocessing steps were resizing the images to 640x640 and splitting the dataset into training (70%), validation (15%), and test (15%) sets. The augmentation was applied directly during training.

3.3 Experimental Results

After conducting 6 different experiments, we achieved the best accuracy with RT-DETR Large (using transfer learning), achieving 0.45 mAP50. The second-best model was YOLO11 Medium (using transfer learning), achieving 0.42 mAP50. Looking at the accuracy table, a clear pattern is visible. The larger the model, the better the accuracy. This result was expected, and we can assume that we could have achieved even higher accuracy had we used larger models.

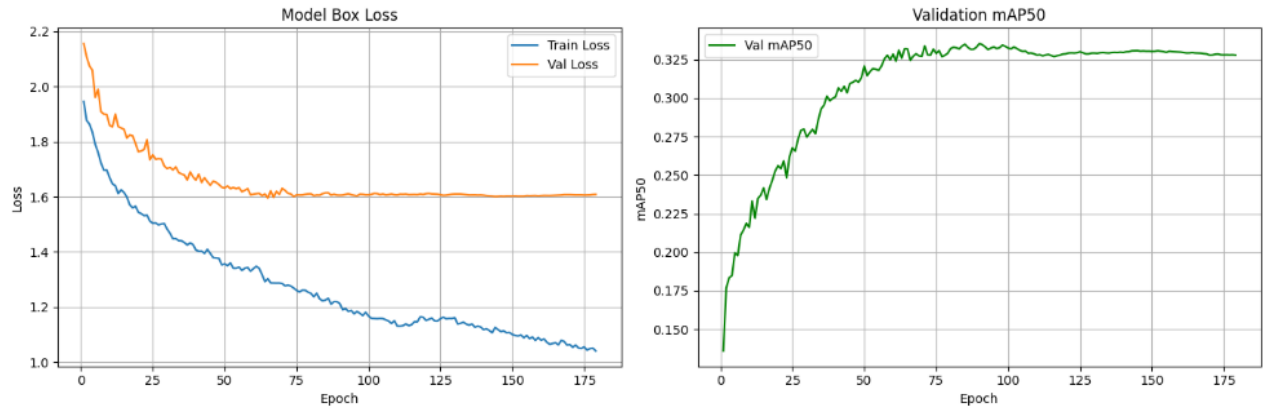
CNN Architecture	Train mAP50	Validation mAP50	Test mAP50
------------------	-------------	------------------	------------

YOLOv5_NANO_TRANSFER	0.74	0.33	0.36
YOLO11_NANO	0.53	0.26	0.26
YOLO11_NANO_TRANSFER	0.73	0.35	0.37
YOLO11_SMALL_TRANSFER	0.71	0.37	0.39
YOLO11_MEDIUM_TRANSFER	0.70	0.38	0.42
RT_DETR_LARGE_TRANSFER	0.88	0.40	0.45



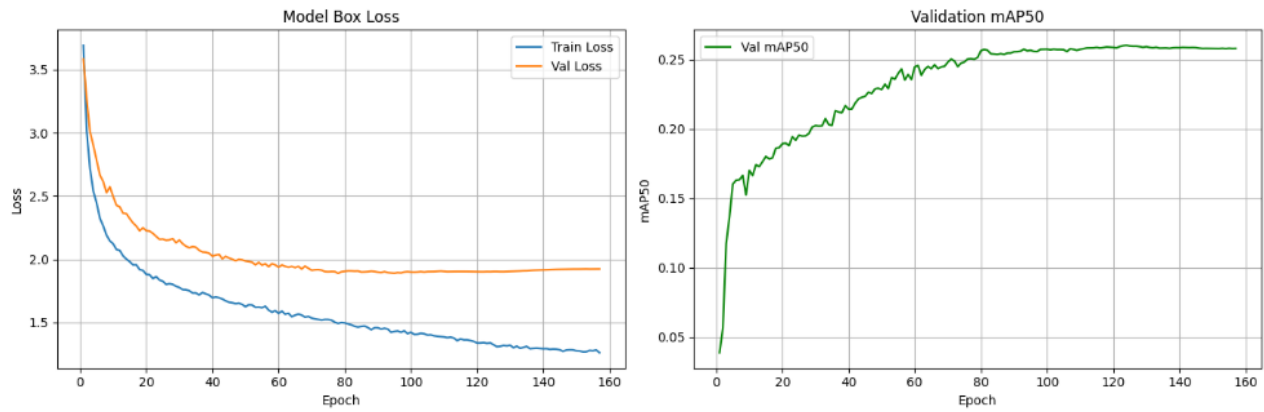
(Graph displaying the mAP50 based on different models)

YOLOv5 Nano (Transfer) Training History



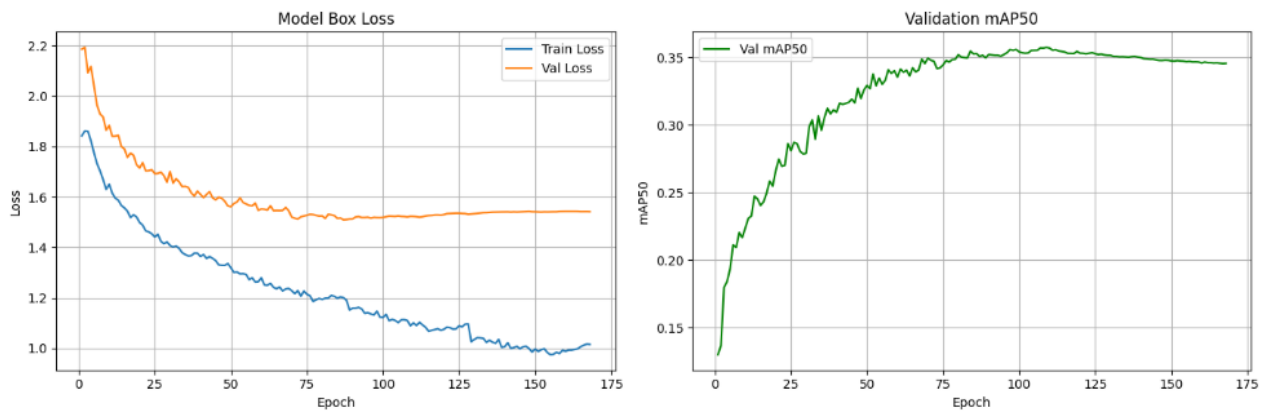
(YOLOv5 Nano (Transfer) Training History)

YOLOv11 Nano (Scratch) Training History



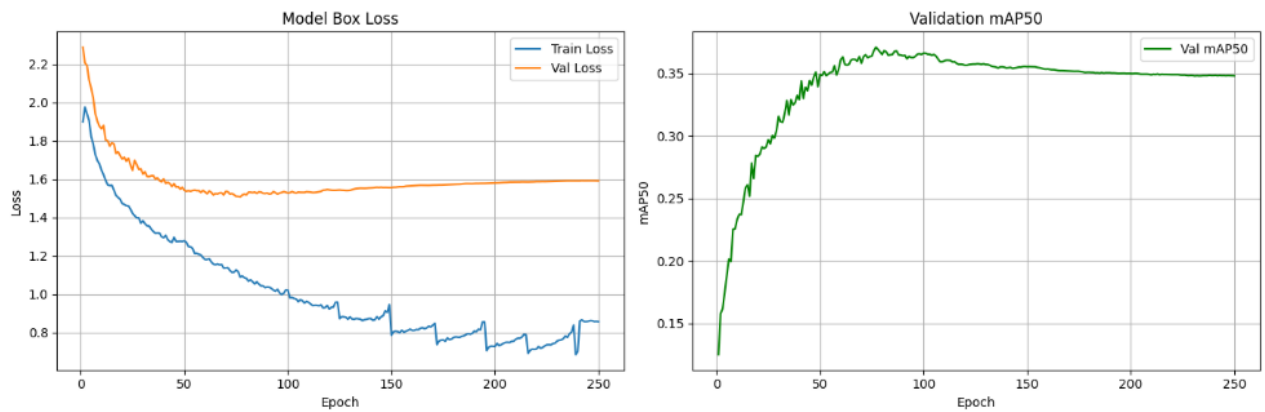
(YOLOv11 Nano (No Transfer) Training History)

YOLOv11 Nano (Transfer) Training History



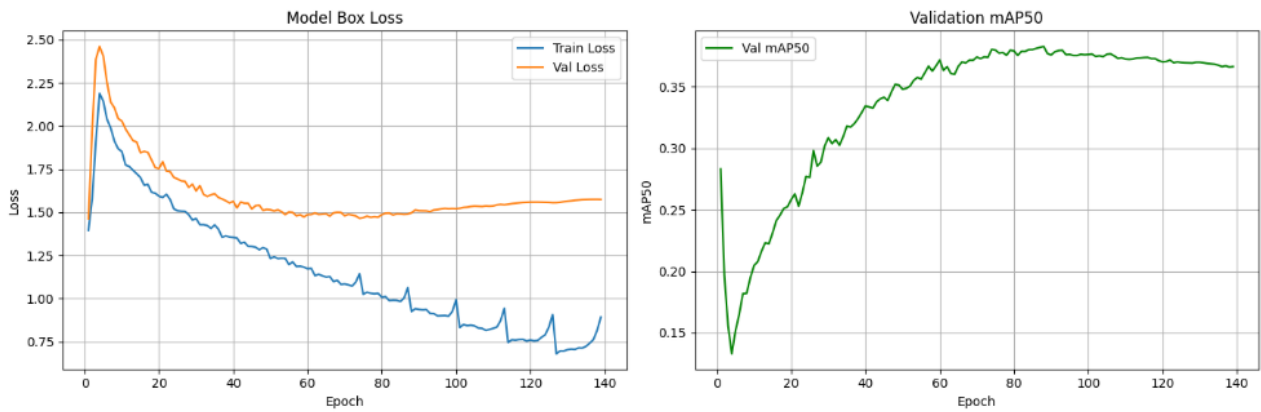
(YOLOv11 Nano (Transfer) Training History)

YOLOv11 Small (Transfer) Training History



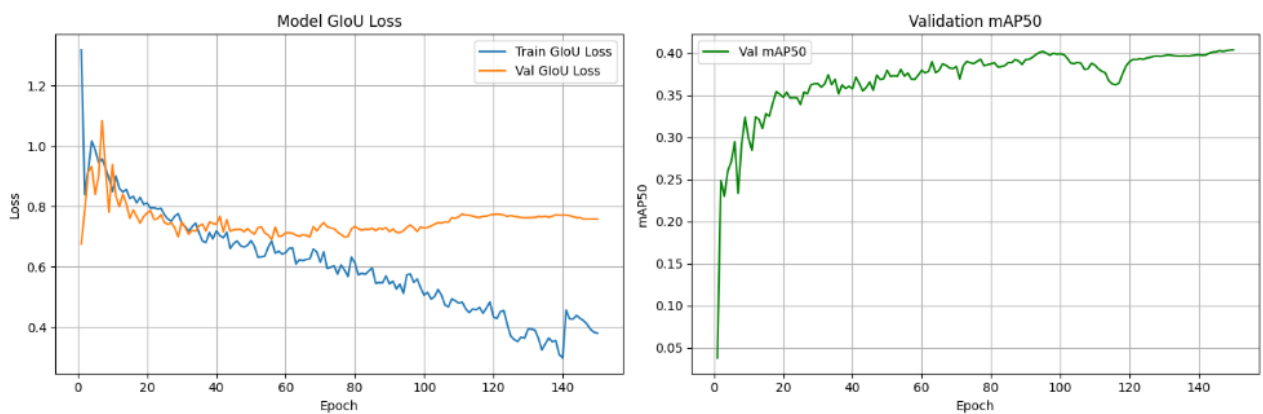
(YOLO11 Small (Transfer) Training History)

YOLOv11 Medium (Transfer) Training History



(YOLO11 Medium (Transfer) Training History)

RT-DETR Large Training History (default hyperparams)



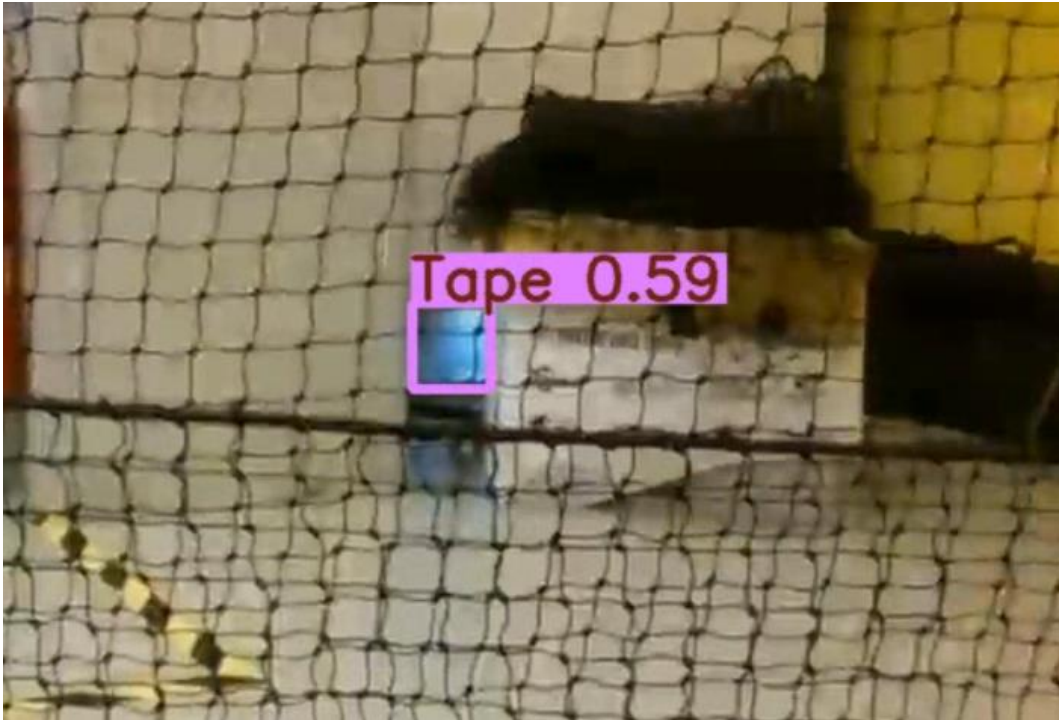
(RT-DETR Large (Transfer) Training History)

3.4 Limitations

There are a few limitations of our applications. First, the model's accuracy is limited by the quality and size of the training dataset and by the size of the object detection model. From our experiments with the drone's live video feed, the drone successfully detects objects from the specified classes, however, it sometimes misclassifies them. For example, we observed the drone classifying the hammer as a screwdriver. Such misclassifications are not a major issue, since the main objective is to detect hazardous objects, and knowing which exact object was detected is a bonus. One other issue with the model, which occurred only very rarely, was detecting a non-existent object. This detection usually occurred only in a single frame, and it wasn't an overall issue. The second limitation was the actual drone. The DJI Tello Edu has very limited controls and no proper sensors. The main problem with the drone was its inconsistent movement. We experimented by sending the same commands multiple times, and the drone performed inconsistent movements. The drone was also unstable and could not remain at a single position when no movement commands were sent. This made it difficult for us to create a consistent automatic movement sequence, since the drone would often drift away from its designated path. Since the drone doesn't have GPS or any other reliable position tracking sensor, we had to use dead-reckoning to keep track of the drone's position. Even though our dead-reckoning approach was accurate in theory, the drone's drifting and inaccurate movement made the position tracking slightly inaccurate, and it worsened over time as the errors accumulated.



(Example of misclassified image captured from the drone's video stream)



(Example of detecting a non-existent object captured from the drone's video stream)

4 Discussion and Conclusions

In conclusion, we built a machine vision application that uses a drone that navigates autonomously, and by using its stream, we can detect and report hazardous objects. Even though our project is only a proof of concept, it achieved promising results, enabling us to detect and report objects with decent accuracy. The next steps for improving our application are straightforward. First, to improve the model's accuracy, we would need larger, higher-quality datasets. Even though we have between 1500 and 2500 images per class, the objects are often very small, and even within the same class, they differ too much in shape and colour, which makes it harder for the model to train properly. Second, by using larger models, we could improve accuracy even further, however, this comes at the cost of slower inference time, so a computer with a high-end GPU would be needed to run the application. Third, if the application were used in production, it would be very helpful to add custom images to the dataset taken directly at the work area by the drone. This step alone could significantly improve accuracy. Finally, using a more specialized drone with better sensors and a higher-resolution camera would allow for a more optimized flight path, better object detection accuracy, and better object mapping estimation, either via GPS or other sensors.

5 References

- Shao, S., Li, Z., Zhang, T., Peng, C., Yu, G., Zhang, X., Li, J. and Sun, J. (2019) 'Objects365: A Large-scale, High-quality Dataset for Object Detection', Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), pp. 8430-8439. Available at: https://openaccess.thecvf.com/content_ICCV_2019/papers/Shao_Objects365_A_Large-Scale_High-Quality_Dataset_for_Object_Detection_ICCV_2019_paper.pdf (Accessed 23 May 2026).
- Mateo Forero (2023) 'Spill Detection Computer Vision Model', Roboflow Universe. Available at: <https://universe.roboflow.com/mateo-forero-uao1f/spill-detection-sh5mc> (Accessed: 23 May 2026).
- Zhao, Y., Lv, W., Xu, S., Wei, J., Wang, G., Dang, Q., Liu, Y. and Chen, J., (2023) 'DETRs Beat YOLOs on Real-time Object Detection', arXiv:2304.08069. Available at: <https://arxiv.org/abs/2304.08069> (Accessed: 24 May 2026).
- Redmon, J., Divvala, S., Girshick, R. and Farhadi, A. (2015) 'You Only Look Once: Unified, Real-Time Object Detection', arXiv:1506.02640. Available at: <https://arxiv.org/abs/1506.02640> (Accessed: 24 May 2026).
- Ryze Technology (2019) 'Tello EDU', Available at: <https://www.ryzerobotics.com/tello-edu> (Accessed: 24 May 2026).
- Roboflow (2026) Roboflow Universe, Available at: <https://universe.roboflow.com/> (Accessed: 24 May 2026).
- Duba, O., Ruiz, H.F., Thollot, F.P.A.L. (2026) 'Deep learning project Drone Dataset', Roboflow Universe, Available at: https://universe.roboflow.com/ondrejs-workspace-tlj5d/deep_learning_project_drone (Accessed: 24 May 2026).
- Sapkota, R. et al. (2025) 'YOLO advances to its genesis: a decadal and comprehensive review of the You Only Look Once (YOLO) series'. ResearchGate, Available at: https://www.researchgate.net/figure/YOLOv11-architecture-diagram-enhanced-%20backbone-with-C3k2-blocks-replacing-C2f-SPPF-for_fig6_392596103 (Accessed: 25 May 2026).
- Khanam, R., Hussain, M. (2024) 'YOLOv11: An Overview of the Key Architectural Enhancements', arXiv:2410.17725v1. Available at: <https://arxiv.org/html/2410.17725v1> (Accessed: 24 May 2026)
- Rath, S.J. (2024) 'RT-DETR: Paper Explanation and Inference', DebuggerCafe, Available at: <https://debuggercafe.com/rt-detr/> (Accessed: 25 May 2026)
- Lin, T.-Y., Maire, M., Belongie, S., Bourdev, L., Girshick, R., Hays, J., Perona, P., Ramanan, D., Zitnick, C.L. and Dollár, P. (2014) 'Microsoft COCO: Common Objects in Context', arXiv:1405.0312, Available at: <https://arxiv.org/abs/1405.0312> (Accessed 25 May 2026)